STARK 증명 시스템 분석

개요

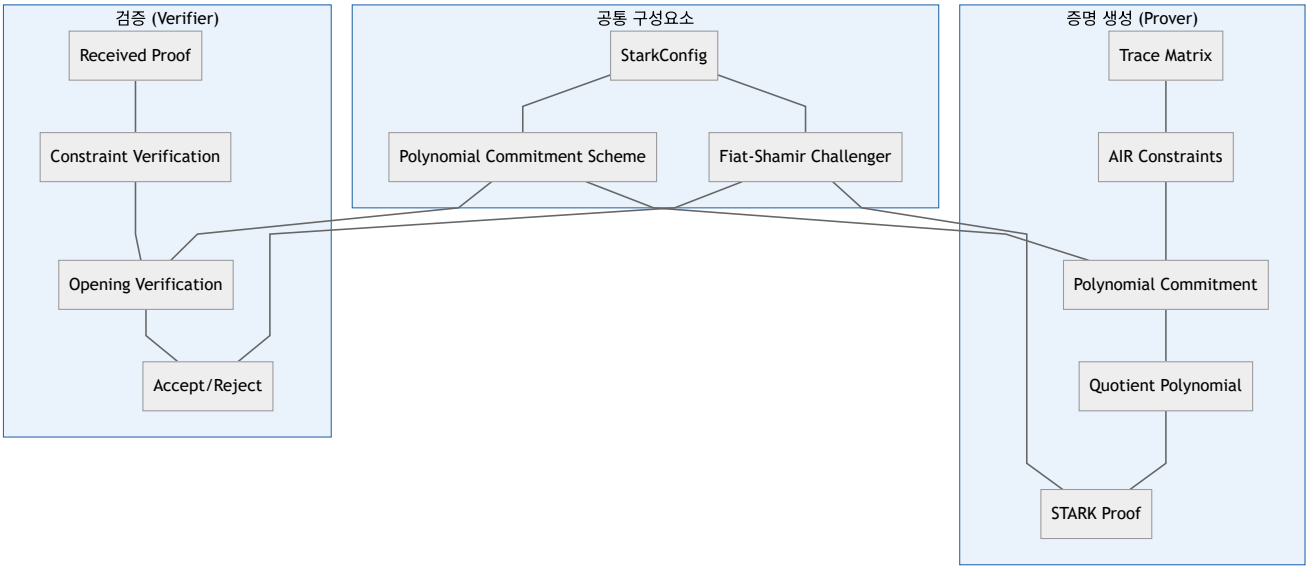
이 문서는 Plonky3의 uni-stark 프레임워크를 통해 구현된 STARK(Scalable Transparent Arguments of Knowledge) 증명 시스템의 전체 구조와 동작 원리를 분석합니다. STARK는 polynomial IOP(Interactive Oracle Proof)를 기반으로 하는 영지식 증명 시스템으로, 확장성과 투명성을 제공하면서도 강력한 보안 보장을 제공합니다.

uni-stark는 단변수(univariate) STARK 구현을 위한 최소한의 프레임워크로, 증명 생성(proving)과 검증(verification) 과정을 체계적으로 구현합니다. 이 시스템은 AIR(Algebraic Intermediate Representation)로 표현된 계산을 다항식 제약조건으로 변환하고, FRI 프로토콜을 통해 이를 효율적으로 증명합니다.

uni-stark 프레임워크 구조

핵심 컴포넌트

uni-stark 프레임워크는 다음과 같은 핵심 컴포넌트들로 구성됩니다:



모듈 구조

```
// uni-stark/src/lib.rs의 주요 모듈들
mod config; // STARK 설정과 매개변수
mod folder; // 제약조건 폴딩 구현
mod proof; // 증명 데이터 구조
mod prover; // 증명 생성 로직
mod verifier; // 증명 검증 로직
mod symbolic_builder; // 심볼릭 제약조건 구축
```

각 모듈은 특정한 역할을 담당하며, 전체 STARK 프로토콜의 구현을 체계적으로 분리합니다.

증명 생성 파이프라인

1. 초기화 및 설정

증명 생성 과정은 `prove` 함수에서 시작됩니다:

```
pub fn prove<SC, A>(
config: &SC,
air: &A,
trace: RowMajorMatrix<Val<SC>>,
public_values: &Vec<Val<SC>>,
) -> Proof<SC>
where
SC: StarkGenericConfig,
A: Air<SymbolicAirBuilder<Val<SC>>> + for<'a> Air<ProverConstraintFolder<'a, SC>
```

주요 초기화 단계:

1. **Trace 매개변수 계산**:

- `degree = trace.height()` : trace의 높이 N = 2^n
- `log_degree = log2_strict_usize(degree)` : $\log_2(N)$
- `log_ext_degree = log_degree + config.is_zk()` : ZK 확장 고려

2. **제약조건 분석**:

- `symbolic_constraints = get_symbolic_constraints(air, 0, public_values.len())`
- 제약조건 개수와 차수 계산
- quotient polynomial의 차수 결정

2. Trace 커밋

```
let (trace_commit, trace_data) =
info_span!("commit to trace data").in_scope(|| pcs.commit([(ext_trace_domain, tr
```

이 단계에서:

- **도메인 확장**: ZK가 활성화된 경우 trace를 2N 크기로 확장
- **다항식 보간**: trace 열들을 다항식으로 보간
- **머클 트리 구축**: 확장된 trace를 머클 트리로 커밋
- **비트 역순 정렬**: 효율적인 FFT를 위한 데이터 재배열

3. Fiat-Shamir 챌린지 생성

```
challenger.observe(Val::<SC>::from_u8(log_ext_degree  as u8));
challenger.observe(Val::<SC>::from_u8(log_degree  as u8));
challenger.observe(trace_commit.clone());
challenger.observe_slice(public_values);
```

```
let  alpha: SC::Challenge = challenger.sample_algebra_element();
```

Fiat-Shamir 변환을 통해:

- **인스턴스 데이터 흡수**: 차수 정보와 공개 값들
- **커밋 흡수**: trace 커밋 해시
- **챌린지 샘플링**: 제약조건 결합을 위한 α 값

4. Quotient Polynomial 계산

```
let  quotient_values = quotient_values(
air,
public_values,
trace_domain,
quotient_domain,
trace_on_quotient_domain,
alpha,
constraint_count,
);
```

핵심 계산 과정:

1. **제약조건 평가**: 각 quotient domain 점에서 제약조건 C(x) 계산
2. **선형 결합**: α를 사용하여 모든 제약조건을 하나로 결합
3. **Quotient 계산**: `Q(x) = C(x) / Z_H(x)`
4. **병렬 처리**: `maybe-rayon` 을 통한 효율적인 병렬 계산

5. Quotient Polynomial 커밋

```
let quotient_flat = RowMajorMatrix::new_col(quotient_values).flatten_to_base();
let (quotient_commit, quotient_data) = info_span!("commit to quotient poly chunk
.in_scope(|| pcs.commit_quotient(quotient_domain, quotient_flat, quotient_degree
```

Quotient polynomial 처리:

- **확장 필드 평탄화**: 확장 필드 원소를 기저 필드 벡터로 변환
- **청크 분할**: 큰 차수의 다항식을 여러 작은 다항식으로 분할
- **개별 커밋**: 각 청크를 독립적으로 커밋

6. 개방 증명 생성

```
let zeta: SC::Challenge = challenger.sample_algebra_element();
let zeta_next = trace_domain.next_point(zeta).unwrap();

let (opened_values, opening_proof) = info_span!("open").in_scope(|| {
let round1 = (&trace_data, vec![vec![zeta, zeta_next]]);
let round2 = (&quotient_data, vec![vec![zeta]; quotient_degree]);
let rounds = [round1, round2].into_iter().collect();
pcs.open(rounds, &mut challenger)
});
```

개방 과정:

- **도메인 외부 점 선택**: $\zeta \notin$ trace_domain
- **다중 점 개방**: trace는 ζ와 $\zeta \cdot h$에서, quotient는 ζ에서 개방

- **FRI 증명**: PCS를 통한 개방 증명 생성

검증 과정

1. 증명 구조 검증

```
let  valid_shape = opened_values.trace_local.len() == air_width
&& opened_values.trace_next.len() == air_width
&& opened_values.quotient_chunks.len() == quotient_degree
&& opened_values.quotient_chunks.iter().all(|qc| qc.len() == SC::Challenge::DIME
```

검증자는 먼저 받은 증명의 구조적 유효성을 확인합니다.

2. 트랜스크립트 재구성

```
challenger.observe(Val::<SC>::from_usize(proof.degree_bits));
challenger.observe(Val::<SC>::from_usize(proof.degree_bits - config.is_zk()));
challenger.observe(commitments.trace.clone());
challenger.observe_slice(public_values);

let  alpha = challenger.sample_algebra_element();
challenger.observe(commitments.quotient_chunks.clone());

let  zeta = challenger.sample_algebra_element();
```

증명자와 동일한 순서로 Fiat-Shamir 트랜스크립트를 재구성합니다.

3. 개방 검증

```
pcs.verify(coms_to_verify, opening_proof, &mut challenger)
.map_err(VerificationError::InvalidOpeningArgument)?;
```

PCS를 통해 커밋된 다항식들의 개방 증명을 검증합니다.

4. 제약조건 검증

```
let mut folder = VerifierConstraintFolder {
main,
public_values,
is_first_row: sels.is_first_row,
is_last_row: sels.is_last_row,
is_transition: sels.is_transition,
alpha,
accumulator: SC::Challenge::ZERO,
};
air.eval(&mut folder);
let folded_constraints = folder.accumulator;

if folded_constraints * sels.inv_vanishing != quotient {
return Err(VerificationError::OodEvaluationMismatch);
```

```
}
```

최종적으로 `C(ζ) / Z_H(ζ) = Q(ζ)` 등식을 확인합니다.

Challenger와 Fiat-Shamir 변환

DuplexChallenger 구조

```
pub struct DuplexChallenger<F, P, const WIDTH: usize, const RATE: usize> {
pub sponge_state: [F; WIDTH],
pub input_buffer: Vec<F>,
pub output_buffer: Vec<F>,
pub permutation: P,
}
```

동작 원리

1. **흡수 단계 (Absorbing)**:

```
fn observe(&mut self, value: F) {
self.output_buffer.clear();
self.input_buffer.push(value);
if self.input_buffer.len() == RATE {
self.duplexing();
}
```

```
}
```

2. **Duplexing 과정**:

```
fn duplexing(&mut self) {
// 입력을 sponge state의 rate 부분에 복사
for (i, val) in self.input_buffer.drain(..).enumerate() {
self.sponge_state[i] = val;
}
// 암호학적 순열 적용
self.permutation.permute_mut(&mut self.sponge_state);
// 출력 버퍼에 rate 부분 복사
self.output_buffer.clear();
self.output_buffer.extend(&self.sponge_state[..RATE]);
}
```

3. **압축 단계 (Squeezing)**:

```
fn sample(&mut self) -> EF {
if !self.input_buffer.is_empty() || self.output_buffer.is_empty() {
self.duplexing();
}
self.output_buffer.pop().expect("Output buffer should be non-empty")
}
```

보안 고려사항

1. **Capacity 보안**: `WIDTH - RATE` 크기의 capacity가 보안 수준 결정
2. **순열 강도**: 암호학적 순열의 안전성이 전체 시스템 보안에 직결
3. **도메인 분리**: 서로 다른 용도의 챌린지는 별도 도메인에서 생성

제약조건 폴딩 (Constraint Folding)

ProverConstraintFolder

증명자 측에서 사용되는 제약조건 폴더:

```
pub struct ProverConstraintFolder<'a, SC: StarkGenericConfig> {
pub main: RowMajorMatrixView<'a, PackedVal<SC>>,
pub public_values: &'a Vec<Val<SC>>,
pub is_first_row: PackedVal<SC>,
pub is_last_row: PackedVal<SC>,
pub is_transition: PackedVal<SC>,
pub alpha_powers: &'a [SC::Challenge],
pub decomposed_alpha_powers: &'a [Vec<Val<SC>>],
pub accumulator: PackedChallenge<SC>,
pub constraint_index: usize,
}
```

제약조건 누적

```
fn assert_zero<I: Into<Self::Expr>>(&mut self, x: I) {
let alpha_power = self.alpha_powers[self.constraint_index];
self.accumulator += Into::<PackedChallenge<SC>>::into(alpha_power) * x.into();
self.constraint_index += 1;
}
```

각 제약조건 `C_i` 에 대해 `α^i * C_i` 를 누적하여 최종적으로 `Σ α^i * C_i` 를 계산합니다.

VerifierConstraintFolder

검증자 측에서는 더 간단한 구조를 사용:

```
pub struct VerifierConstraintFolder<'a, SC: StarkGenericConfig> {
pub main: ViewPair<'a, SC::Challenge>,
pub public_values: &'a Vec<Val<SC>>,
pub is_first_row: SC::Challenge,
pub is_last_row: SC::Challenge,
pub is_transition: SC::Challenge,
pub alpha: SC::Challenge,
pub accumulator: SC::Challenge,
}
```

증명 데이터 구조

Proof 구조

```
pub struct Proof<SC: StarkGenericConfig> {
pub(crate) commitments: Commitments<Com<SC>>,
pub(crate) opened_values: OpenedValues<SC::Challenge>,
pub(crate) opening_proof: PcsProof<SC>,
pub(crate) degree_bits: usize,
}
```

Commitments

```
pub struct Commitments<Com> {
pub(crate) trace: Com, // Trace 커밋
pub(crate) quotient_chunks: Com, // Quotient 청크들의 커밋
pub(crate) random: Option<Com>, // ZK용 랜덤 다항식 커밋
}
```

OpenedValues

```
pub struct OpenedValues<Challenge> {
pub(crate) trace_local: Vec<Challenge>, // ζ에서의 trace 값들
pub(crate) trace_next: Vec<Challenge>, // ζ·h에서의 trace 값들
pub(crate) quotient_chunks: Vec<Vec<Challenge>>, // ζ에서의 quotient 청크 값들
pub(crate) random: Option<Vec<Challenge>>, // ZK용 랜덤 값들
```

```
}
```

성능 최적화

병렬 처리

```
(0..quotient_size)
.into_par_iter()
.step_by(PackedVal::<SC>::WIDTH)
.flat_map_iter(|i_start| {
// 병렬로 quotient 값들 계산
})
.collect()
```

`maybe-rayon` 을 통해 조건부 병렬 처리를 지원합니다.

SIMD 최적화

- **PackedVal**: 여러 필드 원소를 하나의 SIMD 레지스터에 패킹
- **PackedChallenge**: 확장 필드 원소의 SIMD 처리
- **벡터화된 연산**: 제약조건 평가의 벡터화

메모리 최적화

- **스트리밍 처리**: 큰 데이터를 청크 단위로 처리
- **제로 카피**: 불필요한 데이터 복사 최소화

- **효율적인 버퍼링**: 입출력 버퍼의 최적 관리

보안 분석

Soundness 오류

1. **제약조건 결합**: `n/|EF|` (n = 제약조건 개수)
2. **도메인 외부 평가**: `dN/|EF|` (d = 제약조건 차수, N = trace 길이)
3. **FRI 프로토콜**: FRI의 고유한 soundness 보장

Completeness 오류

- **도메인 충돌**: `|gK|/|EF| = 2N/|EF|` (ζ가 quotient domain에 속할 확률)

Zero-Knowledge

ZK 모드에서:

- **랜덤 다항식**: trace와 동일한 차수의 랜덤 다항식 추가
- **통계적 ZK**: 완벽한 ZK가 아닌 통계적 ZK 제공
- **랜덤성 검증**: 랜덤 커밋의 존재 여부 확인

실제 사용 예제

기본 증명 생성

```
// 설정 초기화
let config = StarkConfig::new(pcs, challenger);
```

```
// AIR와 trace 준비
let air = MyAir::new();
let trace = generate_trace();
let public_values = vec![];


// 증명 생성
let proof = prove(&config, &air, trace, &public_values);


// 증명 검증
let result = verify(&config, &air, &proof, &public_values);
```

해시 함수 증명

예제에서 볼 수 있는 실제 사용 사례:

```
// Blake3 증명
let proof_goal = ProofObjective::Blake3(Blake3Air {});
let result = prove_monty31_keccak(proof_goal, dft, num_hashes);


// Poseidon2 증명
let p2_air = VectorizedPoseidon2Air::new(constants);
let proof_goal = ProofObjective::Poseidon2(p2_air);
let result = prove_monty31_poseidon2(proof_goal, dft, num_hashes, perm16, perm2
```

결론

Plonky3의 uni-stark 프레임워크는 STARK 증명 시스템의 완전한 구현을 제공합니다. 주요 특징:

1. **모듈화된 설계**: 각 컴포넌트가 명확히 분리되어 유지보수와 확장이 용이
2. **효율적인 구현**: SIMD 최적화와 병렬 처리를 통한 고성능
3. **유연한 설정**: 다양한 필드와 해시 함수 지원
4. **강력한 보안**: 이론적으로 검증된 보안 보장
5. **실용적 사용**: 실제 암호학적 프리미티브의 증명 가능

이 시스템은 확장 가능하고 투명한 영지식 증명의 실용적 구현을 제공하며, 다양한 응용 분야에서 활용될 수 있는 견고한 기반을 제공합니다.